



Getting Started With CUDA C on an Nvidia Jetson: Hello CUDA World!

Mohammed Billoo • March 13, 2024

INTRODUCTION

Sales of Graphics Processing Units (GPU) have exploded over the past few years due to the rising popularity of machine learning applications. However, GPUs were originally architected and designed for gaming applications. Their native architecture naturally translated into use in machine learning due to the similar operations performed. Nvidia is one of the most popular GPU vendors. Their CUDA framework is used to leverage the underlying GPU hardware to efficiently implement gaming, and now, machine learning applications. In this series of blog posts, I will walk you through the underlying GPU architecture and how it enables efficient implementation of gaming and machine learning applications. Additionally, I will also walk you through examples written in C using the CUDA framework. For each application, I will compare the performance of a GPU-based implementation against a CPU-based implementation to demonstrate the efficiencies of a GPU.

Just with any first embedded software project, we will start with a "Hello World" example. However, in the case of GPU applications, the equivalent of "Hello World" is adding an array (or vector) of two floating-point arrays.

GETTING STARTED

We will be using the Jetson Nano development kit (<https://developer.nvidia.com/buy-jetson?product=je...>), which is a great introductory platform priced at 149USD. After procuring the development kit, you will need to download the Nvidia SDK manager (<https://developer.nvidia.com/sdk-manager>) and follow the steps to install Ubuntu and the CUDA framework on the Jetson Nano.

Please note that only **Ubuntu 18.04** is supported on the Jetson Nano, and thus will require the same version of Ubuntu on the host PC. I was able to use VMWare Workstation Player **16** (not 17), which can be downloaded

here: <https://customerconnect.vmware.com/en/downloads/in....> Also note that there are issues when using VirtualBox to flash the Jetson Nano.

You can retrieve the source code associated with this blog post series from here: <https://github.com/mabembedded/er-cuda-c>. We will be focusing on the "hello-cuda-world" directory.

The following are two high-level distinctions between CUDA-C applications and traditional C applications:

- Source files for CUDA-C applications have a ".cu" extension, whereas source files for traditional C applications have a ".c" extension.
- CUDA-C applications are compiled using the "nvcc" compiler whereas traditional C applications are compiled using the "gcc" compiler.

If we navigate to the hello-cuda-world directory in the above repository, we can see the following two files:

- **hello-cuda-world.cu**: This is the CUDA-C (i.e. GPU-based) implementation of the array addition application.
- **hello-cuda-world_cpu.c**: This is the traditional (i.e. CPU-based) implementation of the array addition application.

UNDERSTANDING AND COMPILING THE APPLICATION

Let us open and review the "hello-cuda-world_cpu.c" source file to understand the traditional implementation first. Then we can understand how the CUDA implementation leverages the GPU to improve the performance of the same application at scale. The following listing shows the source file of the traditional C implementation:

```
#include <stdlib.h>
int main()
{
    long int N = 10000;
    int size = N * sizeof(float);
    long int i = 0;
    float* h_A = (float*)malloc(size);
    float* h_B = (float*)malloc(size);
    float* h_C = (float*)malloc(size);
    for (i = 0; i < N; i++) {
        h_A[i] = i*0.5;
        h_B[i] = i*0.25;
    }
}
```

```

    for (i = 0; i < N; i++) {
        h_C[i] = h_A[i] + h_B[i];
    }
    return 0;
}

```

As we can see above, the application performs the following tasks:

- Allocates memory for three floating point arrays with 10000 elements.
- Fills in two of the three arrays with floating point values.
- Adds the two arrays and stores the results in the third array using a for loop.

After we clone the repository and change the directory to the "hello-cuda-world" directory, we can compile the application by executing the following command on the Jetson Nano:

```
$ gcc -o hello-cuda-world_cpu hello-cuda-world_cpu.c
```

Now, let's walk through the key elements of the CUDA-C implementation.

First, let's start with the top of main:

```

long int N = 10000;
size_t size = N * sizeof(float);
long int i = 0;
// Allocate input vectors h_A and h_B in host memory
float* h_A = (float*)malloc(size);
float* h_B = (float*)malloc(size);
float* h_C = (float*)malloc(size);
for (i = 0; i < N; i++) {
    h_A[i] = i*0.5;
    h_B[i] = i*0.25;
}

```

This is no different from our traditional C implementation above. Here, we allocate memory for three floating-point arrays with 10000 elements and fill in these arrays with values.

The following listing shows the next section in main:

```

float* d_A;
cudaMalloc(&d_A, size);
float* d_B;
cudaMalloc(&d_B, size);
float* d_C;
cudaMalloc(&d_C, size);
// Copy vectors from host memory to device memory
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

```

Here, we are allocating memory for three more floating point arrays. However, we are using the memory in the GPU (via the `cudaMalloc` function) instead of the CPU RAM (via the `malloc` function). Then, we copy the arrays that are in CPU RAM into the GPU via the `cudaMemcpy` function. Unlike the traditional `memcpy` function, which takes three arguments, `cudaMemcpy` takes one additional argument, which instructs the CUDA framework the direction to transfer memory contents. In this instance, we are asking CUDA to transfer memory from the CPU to the GPU via the `cudaMemcpyHostToDevice` directive.

Then, we invoke "VecAdd", which is a "kernel" function executed efficiently across all cores in the GPU. The kernel function and its invocation is shown in the following listing:

```
__global__ void VecAdd(float* A, float* B, float* C, int N)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N)
        C[i] = A[i] + B[i];
}
int main()
{
    .
    .
    .

    // Invoke kernel
    VecAdd<<<1, 100>>>(d_A, d_B, d_C, N);

    .
    .
    .
}
```

The "`__global__`" directive instructs the CUDA framework that the function is meant to be executed on the GPU. This function is known as a "kernel" function. As you can see above, the "VecAdd" function has variables named "blockDim," "blockIdx," and "threadIdx" which aren't defined in the function or the entire application! These variables have special meaning which we will cover in a future blog post.

Later in main, we see that the kernel function is invoked. However, we can see that the invocation format is not the same as a traditional C function. Again, we will learn about the parameters and the invocation of a kernel function in a future blog post.

Finally, the following listing shows how memory content is copied from the GPU back to the CPU RAM:

```
// Copy result from device memory to host memory
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
// Free device memory
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
```

Here, we see the same "cudaMemcpy" function. However, in this instance, we are copying from the GPU to the CPU RAM via the "cudaMemcpyDeviceToHost" directive. Finally, we free up the GPU memory using the "cudaFree" function.

We can execute the following command to compile the CUDA C application (remember, this must be done on the Jetson Nano!):

```
$ nvcc -o hello-cuda-world hello-cuda-world.cu
```

COMPARING PERFORMANCE

We can use the Linux "time" utility to compare the performance of both implementations. For example, we can execute the following function to determine how long our traditional C implementation took to perform the array addition:

```
$ time ./hello-cuda-world_cpu
real    0m0.014s
user    0m0.000s
sys     0m0.012s
```

The "real" value reported is our main concern. Here we see that the application took 14 milliseconds to execute. If we perform the same experiment using the CUDA application, we can see the following result:

```
$ time ./hello-cuda-world
real    0m0.330s
user    0m0.056s
sys     0m0.116s
```

We can see that the CUDA implementation took 330 milliseconds, which is more than 10x longer than the calculation on the CPU!

Why? Didn't we expect that the GPU implementation to result in performance improvement? Why are we seeing **worse** performance. As another experiment, let's set the number of elements in the array to 100 million.

If we recompile the traditional C implementation (after increasing N) and use the time utility to evaluate how long the application took, we see the following result:

```
$ time ./hello-cuda-world_cpu
real    0m3.599s
user    0m2.972s
sys     0m0.592s
```

Similarly, if we recompile the CUDA implementation and use the time utility again, we get the following result:

```
$ time ./hello-cuda-world

real    0m4.645s
user    0m2.264s
sys     0m2.028s
```

This time, although the results are closer, we are still seeing that the GPU implementation is **worse** than the CPU implementation. We will understand why we are seeing this result in the next blog post and will work on an example that truly takes advantage of the GPU's resources.

SUMMARY

In this blog post, I introduced CUDA, which is a framework designed to allow developers to take advantage of Nvidia's GPU hardware acceleration to efficiently implement certain type of applications. We saw how we would implement a vector addition example using CUDA C and compared it against the traditional implementation in "regular" C. We observed that the CUDA C implementation actually performs **worse** than the traditional C implementation (on a CPU). In a future blog post, we will see an example of where the GPU and CUDA really shine.

Previous post by Mohammed Billoo:

 [Getting Started With Zephyr: Bluetooth Low Energy](#)